

Project - Low Level Design

on

Autonomous AI Research Agent

Course Name: Agentic AI

Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	ANIKET JAT	EN22CS304010
2	HIMANSHU SAHRAWAT	EN22CS304029
3	SHRIYANSH GUHA	EN22CS303054
4	ROUNAK PATHEKAR	EN22CS303046
5	YASHASVI BAKORE	EN22CS303059

Group Name: Group 03D11

Project Number: AAI-29

Industry Mentor Name: Mr. Sir

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: 2025-26

Table of Contents

1. Introduction
 - 1.1 Scope of the Document
 - 1.2 Intended Audience
 - 1.3 System Overview
2. System Design
 - 2.1 Application Design
 - 2.2 Process Flow
 - 2.3 Information Flow
 - 2.4 Components Design
 - 2.5 Key Design Considerations
 - 2.6 API Catalogue
3. Data Design
 - 3.1 Data Model
 - 3.2 Data Access Mechanism
 - 3.3 Data Retention Policies
 - 3.4 Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
 - 7.1 Security Aspects
 - 7.2 Performance Aspects
8. References

1. Introduction

1.1 Scope of the Document

This document provides a high-level design of the Autonomous AI Research Agent system, covering architecture, components, data flow, APIs, and system behavior.

1.2 Intended Audience

- Developers
- System Designers
- Interviewers / Recruiters
- AI/ML Engineers

1.3 System Overview

The system is an AI-powered research agent that:

- Accepts user queries
- Performs real-time web search
- Extracts relevant content
- Generates summarized insights using LLMs
- Stores results for future use

2. System Design

2.1 Application Design

The system follows a modular architecture:

- **Frontend Layer** → Streamlit UI
- **Backend Layer** → Python logic
- **Integration Layer** → Tavily API + HuggingFace
- **Storage Layer** → Local file system

2.2 Process Flow

1. User enters query
2. Query sent to Tavily API
3. Search results retrieved
4. Content aggregated
5. LLM processes text
6. Summary generated
7. Output displayed + stored

2.3 Information Flow

User → UI → Backend → Tavily API → Backend → LLM → Output → Storage

2.4 Components Design

◆ UI Component

- Built using Streamlit
- Handles user input and display

◆ Search Component

- Integrates Tavily API
- Fetches real-time data

◆ Processing Component

- Cleans and merges data

◆ LLM Component

- Uses HuggingFace Transformers
- Generates summaries

◆ Storage Component

- Saves output in text files

2.5 Key Design Considerations

- Modular architecture
- Scalability for future multi-agent support
- Minimal external dependency cost (free APIs)
- Easy integration with additional APIs

2.6 API Catalogue

API	Purpose
Tavily API	Web search & data retrieval
HuggingFace Transformers	Text summarization

3. Data Design

3.1 Data Model

- **Input:** User Query (String)
- **Intermediate:** Raw Web Content
- **Output:** Summarized Text

3.2 Data Access Mechanism

- API calls (Tavily)
- Local file read/write operations

3.3 Data Retention Policies

- Reports stored locally
- Can be extended to DB storage
- No sensitive data stored

3.4 Data Migration

- Future upgrade:
 - Local → Cloud storage
 - File system → Database

4. Interfaces

- CLI Interface (main.py)
- Web Interface (Streamlit app.py)

5. State and Session Management

- Stateless backend
- Session handled by Streamlit
- No persistent user session required

6. Caching

- Currently not implemented
- Future enhancement:
 - Cache API responses
 - Store previous queries

7. Non-Functional Requirements

7.1 Security Aspects

- API keys stored in .env
- No exposure in frontend
- Input validation recommended

7.2 Performance Aspects

- Fast response using lightweight models
- Limited data size for summarization
- Scalable for larger models in future

8. References

- Tavily API Documentation
- HuggingFace Transformers Docs
- Streamlit Documentation